

Actividad 3.3 --- Diseño de Arquitectura de Microservicios

Arquitectura de Software

Taller Práctico --- Refactorización de Monolito

Contexto

En las fases anteriores se identificó que el sistema "**Espagueti de Encuestas**" opera como un monolito acoplado donde:

- El backend concentra múltiples responsabilidades.
- No existe separación clara de dominios.
- El acceso a datos y la lógica de negocio están interconectados.
- El sistema no puede escalar de manera independiente.
- No existe autonomía por módulo funcional.

Aunque en la Actividad 3.1 se aplicó arquitectura por capas, el sistema continúa siendo un monolito desplegable como una sola unidad.

Por esta razón, se propone una evolución hacia una arquitectura basada en microservicios.

5.1 Identificación de Bounded Contexts

Se identifican dos dominios funcionales claramente diferenciados:

- Gestión de encuestas (creación y consulta).
- Registro y conteo de votos.

Estos dominios poseen responsabilidades distintas y diferentes características de carga y escalabilidad.

5.2 Propuesta de Microservicios

[1] survey-service

Responsabilidad: - Crear encuestas. - Consultar encuestas. - Gestionar ciclo de vida de encuestas.

Endpoints: - **POST** /surveys - **GET** /surveys/{id}

Base de datos: - Base de datos propia. - No conoce votos individuales.

[2] voting-service

Responsabilidad: - Registrar votos. - Calcular resultados. - Gestionar conteo.

Endpoints: - `POST /votes` - `GET /results/{surveyId}`

Base de datos: - Base de datos independiente. - No accede directamente a la base de datos del survey-service.

3 api-gateway

Responsabilidad: - Punto único de entrada. - Autenticación JWT. - Rate limiting. - Enrutamiento a microservicios.

Función: - Proxy hacia todos los servicios. - El frontend no llama directamente a los microservicios.

5.5 Componentes de Infraestructura Adicionales

Para soportar una arquitectura de microservicios robusta se consideran:

- **Service Discovery (Eureka / Consul)**
 - Registro dinámico de instancias.
 - Balanceo automático.
 - Tolerancia a fallos.
 - **Config Server Centralizado**
 - Externalizar configuración.
 - Gestionar múltiples ambientes.
 - Centralizar application.yml.
 - **Message Broker (RabbitMQ / Kafka)**
 - Comunicación basada en eventos.
 - **Circuit Breaker (Resilience4j)**
 - Evitar fallos en cascada.
 - Controlar servicios no disponibles.
 - Implementar fallback.
 - **Centralized Logging (ELK Stack)**
 - Monitoreo centralizado.
 - Trazabilidad.
 - Observabilidad distribuida.
-

Beneficios de la Descomposición

La migración a microservicios aporta:

- Escalabilidad independiente.
 - Mejor tolerancia a fallos.
 - Equipos trabajando en paralelo.
 - Evolución independiente de dominios.
 - Mayor resiliencia.
 - Mejor observabilidad.
 - Preparación para cloud-native.
-

Conclusión de la Actividad 3.3

La propuesta de microservicios representa una evolución estratégica del sistema "Espagueti de Encuestas", pasando de un monolito acoplado a una arquitectura distribuida basada en dominios independientes.

Esta transformación mejora la escalabilidad y resiliencia del sistema, estableciendo una base sólida para su crecimiento futuro y adaptación a entornos cloud y arquitecturas modernas.